

Note: While debugging Java code, we can use Stacktrace to find the flow of method calls during normal execution using the `dumpStack()` method in the thread class.

Deadlock

By definition, *deadlock* is a critical situation that occurs in a running Java process, when there is a conflict state where two or more synchronised Java objects depend on locking each other, but cannot, because they themselves are locked by the dependent object.

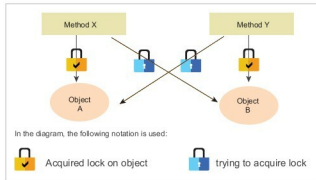


Figure 1: The deadlock situation

In Figure 1, Object A tries to lock Object B while the latter is trying to lock the former. This situation is difficult to debug, because a pre-emptive Java virtual machine can neither detect nor prevent the deadlock.

When the deadlock takes place, the Java program usually hangs. In such a situation, we take a thread dump, whereby we can see the deadlock problem at the end of the thread dump, as shown below:

Found one Java-level deadlock:

=====
"Thread-1":

```
waiting to lock monitor @0091a27c (object 0x140fa790, a
java.lang.Class),
  which is held by "Thread-0"
```

"Thread-0":

```
waiting to lock monitor @0091a25c (object 0x14026800, a
java.lang.Class),
  which is held by "Thread-1"
```

Java stack information for the threads listed above:

=====
"Thread-1":

```
at Deadlock$2.run(Deadlock.java:48)
- waiting to lock <0x140fa790> (a java.lang.Class)
- locked <0x14026800> (a java.lang.Class)
```

"Thread-0":

```
at Deadlock$1.run(Deadlock.java:33)
- waiting to lock <0x14026800> (a java.lang.
Class)
- locked <0x140fa790> (a java.lang.Class)
```

Found 1 deadlock.

Tools like JProfiler, OptimizeIT thread analyser or Samurai are very helpful in analysing a thread dump for any problems and deadlock.

Profiling Java applications

There are specific problems in Java such as high resource consumption—high CPU usage, memory usage, network usage, etc. For these kinds of problems, we have to use the facility of profiling. This measures the performance of the application during runtime by attaching a monitoring tool that gathers vital data on memory usage by each of the objects, and on the instances, the CPU usage and network usage of the running Java application.

For profiling, a lot of tools are available like OptimizeIT, JProfiler, HP-JMeter, etc. OptimizeIT is a very flexible tool that uses the JVMPI facility to profile Java applications. Along with a thread analyser, this tool offers a comprehensive set of features.

JProfiler is an easy-to-use tool for profiling/monitoring remote applications and consumes less memory. With various reporting and snapshot features, this tool can prove to be a very good offline analyser tool.

When our Java programs use the native coding facility (C or C++) like a JNI call or a native call, we may need to monitor native profiling also. For this, we can make use of tools like Insure++, LTPProf, etc.

Analysing Java programs for troubleshooting is an interesting yet complex task; but it can be made simple by using the proper methods. When writing code, we have to keep in mind the possible problems that can arise during runtime, and implement features like logs, console output, error messages, etc, so that when any problem occurs, we can easily identify and solve it. Also, we have to write the code such that minor problems (like boundary/NULL handling) are bypassed and more details are given to the user when major problems are encountered.

By: Magesh Kasthuri

The author is a senior distinguished member of the technical staff at Wipro. He is an enterprise architect in the BFSI field and can be reached at magesh.kasthuri@wipro.com. He has earlier authored a series of articles on Docker based DevOps in OSFY.